ONSURITY — ENGINEERING — BENGALURU

Software Development Engineer III, Java

A hard senior role, deliberately. Java-first, full-stack when the work needs it. You'll own a domain end to end, make the calls without waiting to be told, and answer for them in production. **We hold this bar and we don't lower it to fill the seat.**

LEVEL	TRACK	EXPERIENCE	LOCATION	STACK
SDE III	Individual contributor	6+ years	Bengaluru	Java · Node · React

ABOUT US

Healthcare benefits shouldn't be a luxury

Onsurity is a fast-growing employee healthcare benefits platform. We give SMEs, start-ups and enterprises flexible healthcare subscriptions – discounted medicine, health checkups, fitness plans, free doctor teleconsultations and insurance, in one membership. Our plans aren't limited to full-time employees; we also cover contractual workers, interns, freelancers and consultants, because inclusion is the whole point.

We've raised a **$45M Series B led by Creaegis**, count cricketing legend **Anil Kumble** as a strategic advisor and investor, featured in season one of Disney+ Hotstar's *The Great Indian Disruptors*, and were named a **Dun & Bradstreet Start-up 50 Trailblazer**. More at onsurity.com – or find us on LinkedIn, Instagram and YouTube.

THE ROLE

What this job actually is

We're hiring a small number of senior engineers who can be handed a business problem and come back with a shipped system.

You'll own a domain – claims, wallet, subscriptions, provider network or payments – and be the person accountable for it. Not just the code: the design, the release, the failures, the cost, and the next engineer who has to change it.

Java and Spring Boot are the centre of gravity, but the boundary of your work is the feature, not the language. Expect to write Node services, ship React screens, run your own Kubernetes deployments and debug your own Kafka consumers rather than filing a ticket and waiting.

The problems arrive unfinished. Insurance partners, regulators, hospitals and finance teams each constrain how a feature has to behave, they don't agree with each other, and a member is relying on the result during a hospital visit. Your job is to turn that into a plan, defend the plan in review, ship it, and keep it standing.

We're small enough that your work is live within days, and consequential enough that a rushed decision is paid for by a dozen engineers and thousands of members. We hire at this level to get both halves right.

THE LEVEL

What SDE III means here

Not years, not lines of code. These six are what the level actually asks you to carry.

SCOPE	A domain, end to end. You set the design and defend it.
AMBIGUITY	The problem arrives as a business goal. You turn it into a plan.
INFLUENCE	Your team's technical direction, and the teams that depend on you.
JUDGEMENT	You make the hard trade-offs and own how they turn out.
CONSEQUENCE	You're the last line of defence. If it breaks in production, it's yours.
MULTIPLIER	The team ships better because you're on it.

YOUR WORK

What you'll own

DESIGN

- Own the **HLD** – service boundaries, data flow, scale, failure modes, and the trade-offs written down where someone can argue with them.
- Own the **LLD** – schemas, class and API contracts, state transitions, concurrency, and the edge cases everyone else discovers in production.
- Take both through review before you build. Design that only exists in your head isn't a design.

BUILD

- Design, build, test and maintain Java services – clean, readable, and documented well enough that someone else can take them over.
- Own the APIs other teams depend on, and keep them stable while the product underneath keeps changing.
- Get money and health data right the first time: correct, traceable, and safe to retry when something fails halfway.
- Make data-model decisions you can live with, and change them safely once real members are on them.

RUN

- Carry your services in production. You'll know something is wrong before a customer tells us.
- Instrument before you ship – dashboards, alerts and error budgets are part of the feature, not a follow-up ticket.
- Debug the hard ones – the slow, the intermittent, the ones that only happen at month-end – and fix the cause, not the symptom.
- Keep things fast and cost-sensible, and back every claim about it with numbers.
- Treat member data as confidential by default. Privacy and security aren't a separate checklist here.

MULTIPLY

- Set technical direction for your domain, and write the short docs that make a decision reviewable a year later.
- Review code quickly and usefully – aimed at what will actually hurt later, not at style.
- Mentor engineers, unblock them, and leave the team's default quality higher than you found it.
- Say no with a reason. Push back on scope, on timelines and on your own earlier decisions when the evidence changes.

THE BAR

NON-NEGOTIABLE

How we expect you to operate

Skills get you the interview. These get you the offer, and they're the things we won't compromise on.

High ownership

You treat the systems you build as your own and drive them from idea to production to long-term upkeep. You instrument them, you watch them, you get paged for them. No handing it over at the merge commit.

Highly motivated and self-driven

A bias toward action and a strong sense of accountability. You find the next most useful thing to do without being assigned it, and you say clearly when something slips.

Extensive AI use

You already use Copilot, Claude or ChatGPT heavily – development, code review, debugging, documentation – and you still read every line you ship, because you're the one accountable for it.

Fast, with a floor under it

You ship in days, not quarters, and you don't buy that speed by skipping tests or leaving the mess for the next person.

Direct and low-ego

You disagree in the design review, not in the retro. You change your mind when someone is right, and you write things down so the team doesn't have to keep asking you.

AI

How we expect you to use AI

AI-assisted development is the default here, not a novelty. We expect it to show up in what you get done.

- Use Copilot, Claude or ChatGPT extensively and daily – scaffolding, tests, refactors, code review, debugging, documentation, migrations.

- Read and own every line you ship. "The AI wrote it" is not an explanation for a production incident.
- Know where it's weak — unfamiliar domain rules, security-sensitive paths, and anything touching money or member health data. Verify those yourself.
- Build the workflow, not just the habit: the prompts, context and repo conventions that make the whole team faster, not only you.
- Have opinions about AI in the product too, not only in the toolchain. We're building it into what members use.

MINIMUM

Minimum qualifications

- **6+ years** of professional software development, the majority of it in Java — depth, not the same year repeated.
- **Strong Java** (8 or higher; 17+ preferred) and real Spring / Spring Boot experience, not tutorial-level familiarity.
- **Production Node.js** — services you built and ran, not scripts.
- **Working React** — you can carry a feature from API to UI without waiting for someone else.
- **System design at both altitudes** — HLD you can defend in a review, and LLD detailed enough to hand to another engineer.
- **Kafka** or similar event-driven systems, run in production rather than prototyped.
- **Kubernetes and containers** — you deploy, debug and scale your own service.
- **Good database judgement** across SQL (MySQL, PostgreSQL) and MongoDB — you can find a slow query and fix it yourself.
- **Observability as a habit** — the metrics, logs, traces, dashboards and alerts on your services were set up by you (Prometheus, Grafana, Sentry, ELK or equivalent).
- **Extensive daily AI use** already built into how you work.
- Experience **designing, shipping and operating** backend services that real users depend on.
- Experience being the **owner and on-call** for something you built.
- **Bachelor's degree** in Computer Science, Engineering or a related field, or equivalent practical experience.
- **Clear written and spoken communication** in a cross-functional team.

PREFERRED

Preferred qualifications

- You've led the design of a project that spanned more than one team.
- You've measurably improved an existing system — reliability, latency or cost — and can show the before and after.
- You've run a production incident end to end and written the postmortem yourself.
- Experience mentoring engineers, or informally leading a small team.
- TypeScript end to end, and React Native or another mobile surface.
- You've built with LLM APIs, not only used AI tooling.
- Payments or fintech background — gateways, refunds, reconciliation.
- Health insurance domain exposure — cashless approvals, claims, renewals.
- Experience handling regulated or sensitive customer data.

OUR STACK

What you'd work with

language	Java 17+, Node.js, TypeScript
backend	Spring Boot, REST APIs, JUnit
frontend	React web, React Native mobile app
data	MySQL, PostgreSQL, MongoDB, Redis
streaming	Kafka
platform	AWS, Docker, Kubernetes, GitHub Actions
observability	Prometheus, Grafana, Sentry, ELK
ai	Copilot, Claude, ChatGPT — in the daily workflow

THE BAR, TESTED

What we'll assess

Six dimensions. We index on judgement and evidence, not on how many technologies you can name.

Coding

Correct, readable code under time pressure — and the restraint to keep the solution as simple as the problem allows.

High-level design

A real Onsurity-shaped problem, designed out loud. Boundaries, data flow, scale, failure cases — and what you'd deliberately cut.

Low-level design

The same problem one level down. Schemas, contracts, state transitions, concurrency, and the edge cases you thought of unprompted.

Debugging

A production symptom with incomplete information. What you check first, how you narrow it, and how you stop it recurring.

Ownership deep-dive

One system you owned. We'll go deep — why those decisions, what broke, what it cost, what you'd do differently.

Bar and collaboration

How you handle disagreement and being wrong, and whether the engineers around you get measurably better.

READ TWICE

This role is probably not for you if

- You want a tightly specified ticket queue. Here you'll often write the plan before you write the code.
- You need a long ramp before you contribute. We'd expect you shipping meaningfully inside your first few weeks.
- You'd rather hand a service off once it's live. Ownership here includes the boring long tail and the 2 a.m. page.
- You're uncomfortable working where a bug can delay someone's hospital admission. That's the seriousness this team runs at.

EQUAL OPPORTUNITY

Onsurity is an equal-opportunity employer. We hire on capability and evidence, and we're committed to an inclusive workplace — the same philosophy behind healthcare plans that cover the people other plans leave out.